

CSE 3204
Formal Language, Automata and Computability

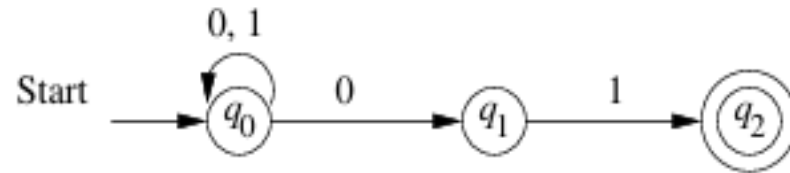


Lecture 2
Nondeterministic Finite Automata

Nondeterministic Finite Automata (NFA)

- Has the ability to be in several states at once.
- For a given state and an input symbol, it can go to multiple states.
- Provide the ability to “guess” something about its input.

Example: Design a NFA that accepts all and only the strings of 0's and 1's that end in 01.



Transition diagram

$(\{q_0, q_1, q_2\}, \{0, 1\}, \delta, q_0, \{q_2\})$

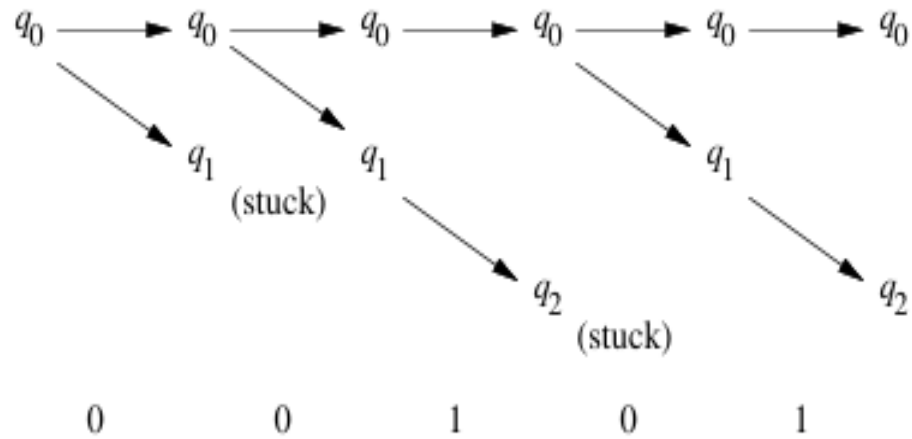
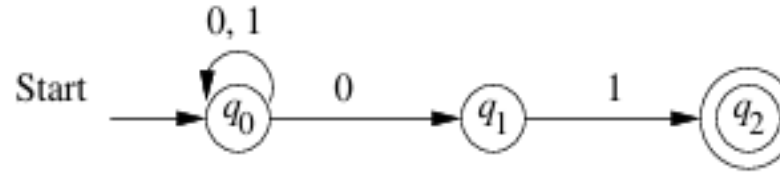
Tuple representation

	0	1
$\rightarrow q_0$	$\{q_0, q_1\}$	$\{q_0\}$
q_1	$\emptyset$	$\{q_2\}$
$*q_2$	$\emptyset$	$\emptyset$

Transition table

Processing of Input String by NFA

- Processing of 00101 by



Nondeterministic Finite Automata (NFA)

- A Nondeterministic Finite Automaton (NFA) consists of:
 - $Q \Rightarrow$ a finite set of states
 - $\Sigma \Rightarrow$ a finite set of input symbols (alphabet)
 - $q_0 \Rightarrow$ a start state in Q
 - $F \Rightarrow$ a subset of Q , is the set of accepting/final states
 - $\delta \Rightarrow$ a transition function that takes a state in Q and an input symbol in Σ as arguments and returns a subset (0, 1 or more states) of Q
- An NFA is defined by the 5-tuple:
 - $(Q, \Sigma, q_0, F, \delta)$

Extended Transition Function

- The extended transition function $\hat{\delta}(q, w)$ returns a set of states that is reached after reading the string w starting from q .
- $\hat{\delta}(q_0, 001) = \{q_0, q_2\}$ for example in slide 3.
- $\hat{\delta}$ is defined in term of δ .

BASIS: $\hat{\delta}(q, \epsilon) = \{q\}$. That is, without reading any input symbols, we are only in the state we began in.

INDUCTION: Suppose w is of the form $w = xa$, where a is the final symbol of w and x is the rest of w . Also suppose that $\hat{\delta}(q, x) = \{p_1, p_2, \dots, p_k\}$. Let

$$\bigcup_{i=1}^k \delta(p_i, a) = \{r_1, r_2, \dots, r_m\}$$

Then $\hat{\delta}(q, w) = \{r_1, r_2, \dots, r_m\}$. Less formally, we compute $\hat{\delta}(q, w)$ by first computing $\hat{\delta}(q, x)$, and by then following any transition from any of these states that is labeled a .

Extended Transition Function

Example 2.8: Let us use $\hat{\delta}$ to describe the processing of input 00101 by the NFA of Fig. 2.9. A summary of the steps is:

1. $\hat{\delta}(q_0, \epsilon) = \{q_0\}$.
2. $\hat{\delta}(q_0, 0) = \delta(q_0, 0) = \{q_0, q_1\}$.
3. $\hat{\delta}(q_0, 00) = \delta(q_0, 0) \cup \delta(q_1, 0) = \{q_0, q_1\} \cup \emptyset = \{q_0, q_1\}$.
4. $\hat{\delta}(q_0, 001) = \delta(q_0, 1) \cup \delta(q_1, 1) = \{q_0\} \cup \{q_2\} = \{q_0, q_2\}$.
5. $\hat{\delta}(q_0, 0010) = \delta(q_0, 0) \cup \delta(q_2, 0) = \{q_0, q_1\} \cup \emptyset = \{q_0, q_1\}$.
6. $\hat{\delta}(q_0, 00101) = \delta(q_0, 1) \cup \delta(q_1, 1) = \{q_0\} \cup \{q_2\} = \{q_0, q_2\}$.

Language of an NFA

- For an NFA $A = (Q, \Sigma, q_0, F, \delta)$, the language of A is

$$L(A) = \{w \mid \hat{\delta}(q_0, w) \cap F \neq \emptyset\}$$

That is, $L(A)$ is the set of strings w in Σ^* such that $\hat{\delta}(q_0, w)$ contains at least one accepting state.

Example 2.9: As an example, let us prove formally that the NFA of Fig. 2.9 accepts the language $L = \{w \mid w \text{ ends in } 01\}$. The proof is a mutual induction of the following three statements that characterize the three states:

1. $\hat{\delta}(q_0, w)$ contains q_0 for every w .
2. $\hat{\delta}(q_0, w)$ contains q_1 if and only if w ends in 0.
3. $\hat{\delta}(q_0, w)$ contains q_2 if and only if w ends in 01.

BASIS: If $|w| = 0$, then $w = \epsilon$. Statement (1) says that $\hat{\delta}(q_0, \epsilon)$ contains q_0 , which it does by the basis part of the definition of $\hat{\delta}$. For statement (2), we know that ϵ does not end in 0, and we also know that $\hat{\delta}(q_0, \epsilon)$ does not contain q_1 , again by the basis part of the definition of $\hat{\delta}$. Thus, the hypotheses of both directions of the if-and-only-if statement are false, and therefore both directions of the statement are true. The proof of statement (3) for $w = \epsilon$ is essentially the same as the above proof for statement (2).

Language of an NFA

INDUCTION: Assume that $w = xa$, where a is a symbol, either 0 or 1. We may assume statements (1) through (3) hold for x , and we need to prove them for w . That is, we assume $|w| = n + 1$, so $|x| = n$. We assume the inductive hypothesis for n and prove it for $n + 1$.

1. We know that $\hat{\delta}(q_0, x)$ contains q_0 . Since there are transitions on both 0 and 1 from q_0 to itself, it follows that $\hat{\delta}(q_0, w)$ also contains q_0 , so statement (1) is proved for w .
2. (If) Assume that w ends in 0; i.e., $a = 0$. By statement (1) applied to x , we know that $\hat{\delta}(q_0, x)$ contains q_0 . Since there is a transition from q_0 to q_1 on input 0, we conclude that $\hat{\delta}(q_0, w)$ contains q_1 .

(Only-if) Suppose $\hat{\delta}(q_0, w)$ contains q_1 . If we look at the diagram of Fig. 2.9, we see that the only way to get into state q_1 is if the input sequence w is of the form $x0$. That is enough to prove the “only-if” portion of statement (2).

Language of an NFA

3. (If) Assume that w ends in 01. Then if $w = xa$, we know that $a = 1$ and x ends in 0. By statement (2) applied to x , we know that $\hat{\delta}(q_0, x)$ contains q_1 . Since there is a transition from q_1 to q_2 on input 1, we conclude that $\hat{\delta}(q_0, w)$ contains q_2 .

(Only-if) Suppose $\hat{\delta}(q_0, w)$ contains q_2 . Looking at the diagram of Fig. 2.9, we discover that the only way to get to state q_2 is for w to be of the form $x1$, where $\hat{\delta}(q_0, x)$ contains q_1 . By statement (2) applied to x , we know that x ends in 0. Thus, w ends in 01, and we have proved statement (3).

Equivalence of DFA and NFA

- Every language that can be described an NFA can also be described by a DFA.
- Subset construction: used to construct a DFA from a given NFA. The constructed DFA can do whatever the corresponding NFA can do.

The subset construction starts from an NFA $N = (Q_N, \Sigma, \delta_N, q_0, F_N)$. Its goal is the description of a DFA $D = (Q_D, \Sigma, \delta_D, \{q_0\}, F_D)$ such that $L(D) = L(N)$. Notice that the input alphabets of the two automata are the same, and the start state of D is the set containing only the start state of N . The other components of D are constructed as follows.

- Q_D is the set of subsets of Q_N ; i.e., Q_D is the *power set* of Q_N . Note that if Q_N has n states, then Q_D will have 2^n states. Often, not all these states are accessible from the start state of Q_D . Inaccessible states can

Subset Construction Continuation

be “thrown away,” so effectively, the number of states of D may be much smaller than 2^n .

- F_D is the set of subsets S of Q_N such that $S \cap F_N \neq \emptyset$. That is, F_D is all sets of N ’s states that include at least one accepting state of N .
- For each set $S \subseteq Q_N$ and for each input symbol a in Σ ,

$$\delta_D(S, a) = \bigcup_{p \text{ in } S} \delta_N(p, a)$$

That is, to compute $\delta_D(S, a)$ we look at all the states p in S , see what states N goes to from p on input a , and take the union of all those states.

	0	1
$\emptyset$	$\emptyset$	$\emptyset$
$\rightarrow \{q_0\}$	$\{q_0, q_1\}$	$\{q_0\}$
$\{q_1\}$	$\emptyset$	$\{q_2\}$
$*\{q_2\}$	$\emptyset$	$\emptyset$
$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$
$*\{q_0, q_2\}$	$\{q_0, q_1\}$	$\{q_0\}$
$*\{q_1, q_2\}$	$\emptyset$	$\{q_2\}$
$*\{q_0, q_1, q_2\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$

Figure 2.12: The complete subset construction from Fig. 2.9

- Section 2.3 of Chapter 2.